

Decorator-driven CLI Command Framework

Professional Command Line Interface (CLI) applications utilize modular architectures to map user inputs to backend functional logic. By implementing a decorator-driven registration system, developers can easily extend the CLI's capabilities while centralizing the complexity of argument parsing and error handling within a unified framework.

Implement a `CLIApp` class that provides a high-level registration decorator and a central execution engine. The framework must support dynamic command registration, handle multiple arguments, and provide informative error messages for unknown commands or signature mismatches (invalid argument counts).

Example 1 Input:

```
RUN greet Alice
```

Output:

```
Hello, Alice!
```

Explanation: The `greet` function is registered as a command. The framework invokes it with the provided argument "Alice" and returns the function's response.

Example 2 Input:

```
RUN add 10 20
```

Output:

```
30
```

Explanation: The `add` command takes two parameters. The framework converts them (if necessary) and executes the logic, returning the sum.

Function Parameter:

- name (str): The keyword used to trigger the command from the CLI.
- args_list (list): The sequence of tokens provided by the user.

Returns

- Any: The output of the registered function.
- str: Error messages for invalid states.

Constraints

- Use a decorator factory `@app.command(name)` for registration.
- Return `"Unknown command: <name>"` for keywords not found in the registry.
- Catch `TypeError` during invocation and return `"Invalid arguments for <name>"`.
- Handle empty inputs by returning `"No command provided"`.

Input Format for Custom Testing Input from stdin contains: `RUN <command_name> <args...>`.

Sample Case 0 Sample Input

```
RUN subtract 10 5
```

Sample Output

```
Unknown command: subtract
```

Explanation The system correctly identifies that the `subtract` command has not been registered and returns the standardized error message.